

LAB 008

DOCKER

CONTAINERS FROM ZERO TO SWARM

Dockerfile • Build • Compose • ECR • Swarm

PART A — GUIDED TUTORIAL

Part 1: What is Docker?

1.1 — The Problem

You've deployed apps by SSHing into EC2 instances, running `yum install`, configuring services, and hoping everything works. But what happens when you move to a different server? A different OS? A teammate's laptop? Things break because the environment is different.

Docker solves this. You package your application **and its entire environment** (OS, libraries, dependencies, config) into a single unit called a **container**. That container runs the same way everywhere, your laptop, an EC2 instance, a Kubernetes cluster.

1.2 — Key Concepts

Concept	Explanation
Image	A read-only blueprint for a container. Like a class in programming. Built from a Dockerfile. Stored in registries (Docker Hub, ECR).
Container	A running instance of an image. Like an object created from a class. Lightweight, isolated, disposable.
Dockerfile	A text file with instructions to build an image. Each instruction creates a layer. Think of it as a recipe.
Layer	Each Dockerfile instruction (FROM, RUN, COPY) creates a layer. Layers are cached, if nothing changed, Docker reuses them. This makes builds fast.
Registry	A place to store and share images. Docker Hub is public. AWS ECR is private. Like GitHub but for Docker images.
Volume	Persistent storage that survives container restarts. Without a volume, data inside a container is lost when it stops.
Docker Compose	A tool for defining multi-container applications in a single YAML file. One command starts everything.

Docker Swarm	Docker's built-in orchestration. Turns multiple Docker hosts into a cluster. Manages scaling, load balancing, and failover.
---------------------	---

1.3 — Docker vs. Virtual Machines

Aspect	Docker Container vs. Virtual Machine
Boot time	Containers: seconds. VMs: minutes.
Size	Containers: MBs. VMs: GBs.
Isolation	Containers share the host OS kernel. VMs run their own OS.
Resource usage	Containers are lightweight, you can run dozens on one machine. VMs are heavy.
Use case	Containers: application packaging and deployment. VMs: full OS isolation.

Part 2: Install Docker

macOS

Step 1: Install Docker Desktop

Download Docker Desktop from <https://www.docker.com/products/docker-desktop/>

Install and launch it. The Docker whale icon should appear in your menu bar.

Verify:

```
docker --version
docker compose version
```

Windows

Step 2: Install Docker Desktop

Download Docker Desktop from <https://www.docker.com/products/docker-desktop/>

During installation, enable WSL 2 (Windows Subsystem for Linux) when prompted.

Restart your computer, then verify:

```
docker --version
docker compose version
```

Linux (Ubuntu/Amazon Linux)

Step 3: Install Docker Engine

```
# Ubuntu
sudo apt update && sudo apt install -y docker.io
sudo systemctl start docker && sudo systemctl enable docker
sudo usermod -aG docker $USER

# Amazon Linux 2023
sudo yum install -y docker
sudo systemctl start docker && sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

Log out and back in (or run `newgrp docker`) for the group change to take effect.

Verify:

```
docker --version
```

Part 3: Your First Container

Let's start by running a container from an existing image. No Dockerfile yet — just pulling and running.

Step 4: Pull and Run Nginx

```
# Pull the official Nginx image
docker pull nginx

# Run it as a container
docker run -d --name my-nginx -p 8080:80 nginx
```

Open your browser and go to <http://localhost:8080>. You should see the Nginx welcome page.

Let's break down the command:

- `-d` — run in detached mode (background)
- `--name my-nginx` — give the container a name
- `-p 8080:80` — map port 8080 on your machine to port 80 inside the container
- `nginx` — the image to use

Step 5: Explore Docker Commands

```
# List running containers
docker ps

# View container logs
docker logs my-nginx

# Execute a command inside the container
docker exec -it my-nginx bash
# (type 'exit' to leave)

# Stop the container
docker stop my-nginx

# Remove the container
docker rm my-nginx

# List all images on your machine
docker images
```

Command	What It Does
---------	--------------

docker ps	List running containers. Add -a to see stopped ones too.
docker logs <name>	View stdout/stderr output from a container.
docker exec -it <name> bash	Open a shell inside a running container. -it = interactive terminal.
docker stop <name>	Gracefully stop a container.
docker rm <name>	Delete a stopped container.
docker rmi <image>	Delete an image from your machine.
docker images	List all images stored locally.

Part 4: Build Your Own Image

Now you'll create a simple Node.js Express API and package it in a Docker image.

Step 6: Create the Project

```
mkdir docker-app && cd docker-app
mkdir src
```

Step 7: Create the Application (src/server.js)

```
const express = require("express");
const app = express();
const PORT = 3000;

app.use(express.json());

let items = [
  { id: 1, name: "Docker", description: "Container platform" },
  { id: 2, name: "Nginx", description: "Web server" },
  { id: 3, name: "Express", description: "Node.js framework" },
];

app.get("/api/items", (req, res) => {
  res.json(items);
});

app.get("/api/health", (req, res) => {
  res.json({ status: "ok", container: require("os").hostname() });
});

app.post("/api/items", (req, res) => {
  const item = { id: items.length + 1, ...req.body };
  items.push(item);
  res.status(201).json(item);
});

app.listen(PORT, () => {
  console.log(`API running on port ${PORT}`);
});
```

Step 8: Create package.json

```
{
  "name": "docker-app",
  "version": "1.0.0",
```

```

    "main": "src/server.js",
    "scripts": {
      "start": "node src/server.js"
    },
    "dependencies": {
      "express": "^4.18.0"
    }
  }
}

```

Step 9: Write the Dockerfile

Create a file called Dockerfile (no extension) in the project root:

```

# Use Node.js 18 as the base image
FROM node:18-alpine

# Set the working directory inside the container
WORKDIR /app

# Copy package.json first (for layer caching)
COPY package.json .

# Install dependencies
RUN npm install --production

# Copy the rest of the application
COPY src/ ./src/

# Expose the port the app runs on
EXPOSE 3000

# Command to run when the container starts
CMD ["npm", "start"]

```

Each line creates a layer. Notice we copy package.json BEFORE the source code — this way, Docker caches the npm install layer and only reruns it if dependencies change.

Note

FROM — base image (alpine = small Linux). WORKDIR: sets the working directory. COPY — copies files from host to container.

RUN — executes a command during build. EXPOSE: documents the port (doesn't publish it). CMD — the command to run at a container start.

Step 10: Build and Run


```
# Build the image (don't forget the dot at the end!)
docker build -t docker-app:v1 .

# Run it
docker run -d --name my-app -p 3000:3000 docker-app:v1

# Test it
curl http://localhost:3000/api/health
curl http://localhost:3000/api/items
```

You should see JSON responses. The health endpoint returns the **container hostname** — this will be useful later when we have multiple containers.

 Tip

If you change your code, rebuild the image: `docker build -t docker-app:v2 .` — Docker reuses cached layers for steps that haven't changed.

Part 5: Docker Compose

Your API works, but in the real world you need multiple services: an API, a database, maybe a frontend. Docker Compose lets you define and run multi-container applications in one file.

Step 11: Add MySQL to the Project

Create a database/ folder with an init script:

```
mkdir database
```

database/init.sql:

```
CREATE DATABASE IF NOT EXISTS docker_lab;
USE docker_lab;

CREATE TABLE items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  description VARCHAR(255)
);

INSERT INTO items (name, description) VALUES
  ('Docker', 'Container platform'),
  ('Nginx', 'Web server'),
  ('Express', 'Node.js framework');
```

Step 12: Update server.js to Use MySQL

Replace src/server.js with:

```

const express = require("express");
const mysql = require("mysql2/promise");
const app = express();
const PORT = 3000;

app.use(express.json());

const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASS || 'secret',
  database: process.env.DB_NAME || 'docker_lab',
  waitForConnections: true,
});

app.get("/api/health", async (req, res) => {
  try {
    await pool.query('SELECT 1');
    res.json({ status: "ok", container: require("os").hostname() });
  } catch (e) {
    res.status(500).json({ status: "error", message: e.message });
  }
});

app.get("/api/items", async (req, res) => {
  const [rows] = await pool.query('SELECT * FROM items');
  res.json(rows);
});

app.post("/api/items", async (req, res) => {
  const { name, description } = req.body;
  const [result] = await pool.query(
    'INSERT INTO items (name, description) VALUES (?, ?)', [name, description]
  );
  res.status(201).json({ id: result.insertId, name, description });
});

app.listen(PORT, () => console.log(`API running on port ${PORT}`));

```

Update package.json to add mysql2:

```

"dependencies": {
  "express": "^4.18.0",
  "mysql2": "^3.6.0"
}

```

Step 13: Create docker-compose.yml

```

services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      DB_HOST: db
      DB_USER: root
      DB_PASS: ${DB_PASSWORD}
      DB_NAME: docker_lab
    depends_on:
      db:
        condition: service_healthy

  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
      MYSQL_DATABASE: docker_lab
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  db_data:

```

Notice the API connects to the database using the service name `db` as the host — Docker Compose creates a network where services can reach each other by name.

Step 14: Create .env File

```
DB_PASSWORD=MyS3cur3P@ss
```

Also create `.env.example` (for the repo):

```
DB_PASSWORD=your_password_here
```

⚠ Warning

Never commit `.env` to Git. Add it to `.gitignore`. Commit `.env.example` instead so others know what variables are needed.

Step 15: Run with Compose

```
# Start everything
docker compose up -d

# Check status
docker compose ps

# View logs
docker compose logs -f

# Test
curl http://localhost:3000/api/health
curl http://localhost:3000/api/items

# Add an item
curl -X POST http://localhost:3000/api/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Compose", "description": "Multi-container tool"}'

# Stop everything
docker compose down
```

The `depends_on` with `service_healthy` ensures the API waits for MySQL to be ready before starting.

 Tip

`docker compose down` removes containers but keeps volumes. Use `docker compose down -v` to also delete volumes (wipes the database).

Part 6: Push to Amazon ECR

Amazon Elastic Container Registry (ECR) is AWS's private Docker registry. You push images here so your EC2 instances and ECS/EKS clusters can pull them.

Step 16: Create an ECR Repository

```
# Create the repository
aws ecr create-repository --repository-name docker-app --region us-east-1
```

Note the repositoryUri from the output — it looks like: 123456789.dkr.ecr.us-east-1.amazonaws.com/docker-app

Step 17: Authenticate Docker with ECR

```
# Login to ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
  123456789.dkr.ecr.us-east-1.amazonaws.com
```

Replace 123456789 with your AWS account ID. You should see "Login Succeeded".

Step 18: Tag and Push

```
# Tag the image for ECR
docker tag docker-app:v1 \
  123456789.dkr.ecr.us-east-1.amazonaws.com/docker-app:v1

# Push to ECR
docker push \
  123456789.dkr.ecr.us-east-1.amazonaws.com/docker-app:v1
```

Go to the AWS Console → ECR → Repositories → docker-app. You should see your image with the v1 tag.

Note

ECR images are private by default. Only your AWS account (or accounts you explicitly grant access) can pull them. This is a key difference from Docker Hub.

PART B — DOCKER SWARM & PRACTICE

Part 7: Docker Swarm on AWS

7.1 — What is Docker Swarm?

So far, you've been running Docker on a single machine. That works for development, but in production you need your app running across multiple servers for high availability, load balancing, and scalability. Docker Swarm is Docker's built-in orchestration tool that turns multiple Docker hosts into a single cluster.

Think of it this way:

- **Without Swarm:** You SSH into each server, run `docker run` manually, and hope nothing crashes.
- **With Swarm:** You tell the swarm "I want 5 copies of my API running" and Swarm figures out which nodes to put them on, restarts them if they crash, and load balances traffic automatically.

7.2 — Swarm Concepts

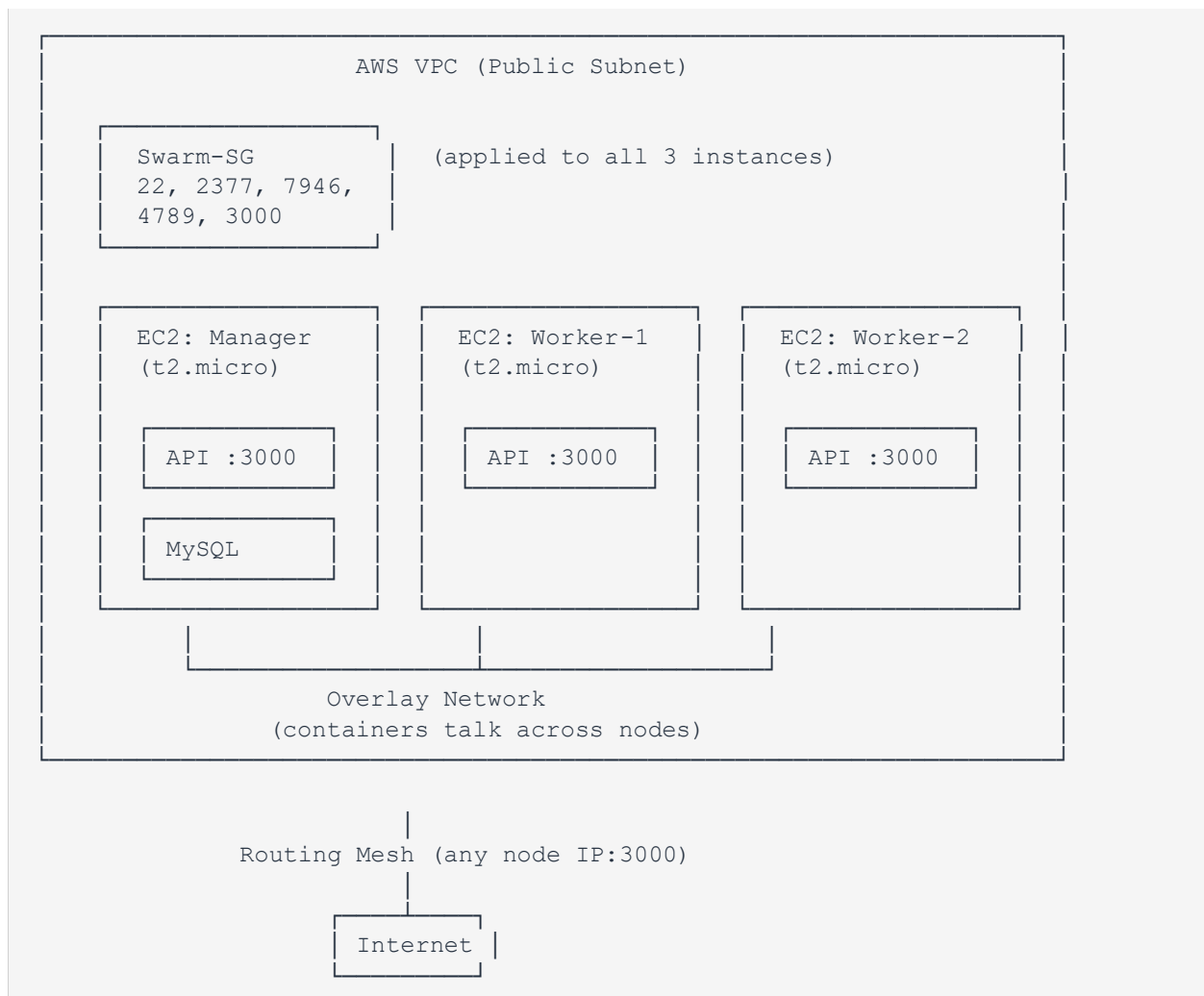
Concept	Explanation
Manager Node	The brain of the swarm. Accepts commands, schedules containers, monitors health. You run docker commands here. A swarm needs at least 1 manager.
Worker Node	The muscle. Receives tasks from the manager and runs containers. Workers don't accept docker service commands — only the manager does.
Service	A definition of what to run: which image, how many replicas, which ports. The swarm equivalent of <code>docker run</code> . Example: "run 3 copies of my API on port 3000".
Task	A single container running as part of a service. If a service has 3 replicas, there are 3 tasks, each running on a node.
Stack	A group of services defined in a <code>docker-compose.yml</code> . You deploy an entire stack with one command (<code>docker stack deploy</code>).
Overlay Network	A virtual network that spans all swarm nodes. Containers on different physical servers can talk to each other as if they were on the same network.

Routing Mesh

Swarm's built-in load balancer. You can hit ANY node's IP on the published port, and Swarm routes the request to a container — even if that container is on a different node.

7.3 — Architecture Diagram

Here is what your Swarm cluster will look like on AWS:



Key points from the diagram:

- **3 EC2 instances** in a public subnet, all in the same security group
- **1 Manager** runs the API + MySQL (database pinned to manager via placement constraint)
- **2 Workers** each run an API container

- **Overlay network** lets all API containers reach MySQL on the manager, even across different EC2 instances
- **Routing mesh** means you can hit ANY of the 3 public IPs on port 3000 and reach an API container

7.4 — Port Mapping: How Traffic Reaches Containers

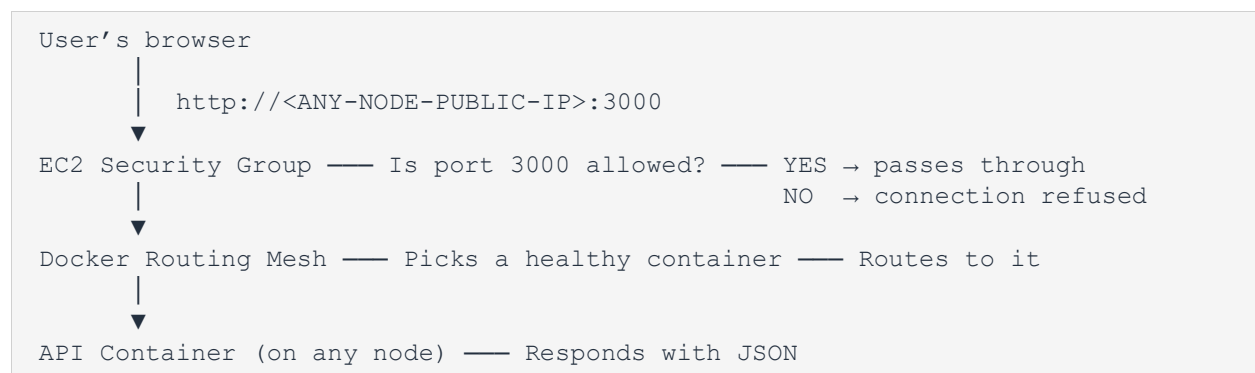
There are two layers of port mapping happening:

Layer 1: EC2 Security Group → EC2 Instance

The security group controls which ports are open on the EC2 instance itself. If port 3000 is not open in the SG, traffic never reaches the instance, Docker doesn't matter.

Layer 2: EC2 Instance → Docker Container

When you publish a port in your Swarm service (ports: "3000:3000"), Docker's routing mesh listens on port 3000 on EVERY node in the swarm. When traffic arrives at any node on port 3000, Swarm routes it to a container running that service — even if the container is on a different node.



Note

This is why the security group is critical. Docker publishes ports, but the EC2 security group is the firewall that sits in front.

If you forget to open port 3000 in the SG, your Swarm services will run fine but nobody can reach them.

7.5 — Security Group: Swarm-SG

Create a single security group for all 3 Swarm nodes. It needs these rules:

Rule	Details
SSH (22) inbound	From: your IP (or 0.0.0.0/0 if no static IP). Purpose: SSH into the instances to manage them.
TCP 2377 inbound	From: Swarm-SG itself (self-referencing). Purpose: Swarm management traffic. This is how the manager and workers communicate cluster state.
TCP 7946 inbound	From: Swarm-SG itself. Purpose: Container network discovery. Nodes use this to find each other and coordinate.
UDP 7946 inbound	From: Swarm-SG itself. Purpose: Same as above but UDP (both protocols needed).
UDP 4789 inbound	From: Swarm-SG itself. Purpose: Overlay network traffic (VXLAN). This is the actual data plane — container-to-container communication across nodes.
TCP 3000 inbound	From: 0.0.0.0/0 (or your IP for testing). Purpose: Your application port. The routing mesh listens on this port on all nodes.
All outbound	To: 0.0.0.0/0. Purpose: Allow instances to reach the internet (download images from ECR, install packages).

⚠ Warning

The Swarm ports (2377, 7946, 4789) must reference the security group itself — meaning traffic is only allowed between Swarm nodes, not from the internet.
Port 3000 is the only port that should be open to the internet (or your IP).

Summary of what each port does:

```
Port 22    → SSH (you → EC2)
Port 2377  → Swarm management API (manager ↔ workers)
Port 7946  → Node discovery + gossip (all nodes ↔ all nodes)
Port 4789  → Overlay network data (container ↔ container across nodes)
Port 3000  → Your app (internet → routing mesh → container)
```

7.6 — Launch 3 EC2 Instances

Launch 3 EC2 instances (t2.micro, Amazon Linux 2023) in a public subnet with the Swarm-SG security group from section 7.5:

- **swarm-manager** — this will be the Swarm manager (runs docker swarm init)
- **swarm-worker-1** — first worker node
- **swarm-worker-2** — second worker node

All 3 should have Docker installed. You can use this user data script:

```
#!/bin/bash
yum update -y
yum install -y docker
systemctl start docker && systemctl enable docker
usermod -aG docker ec2-user
```

7.7 — Initialize the Swarm

SSH into the manager node and initialize the swarm:

```
# On swarm-manager
docker swarm init --advertise-addr <MANAGER-PRIVATE-IP>
```

This outputs a docker swarm join command. Copy it.

7.8 — Join Workers

SSH into each worker and run the join command:

```
# On swarm-worker-1 and swarm-worker-2
docker swarm join --token <TOKEN> <MANAGER-PRIVATE-IP>:2377
```

Back on the manager, verify:

```
docker node ls
```

You should see 3 nodes — 1 Leader and 2 Workers.

7.9 — Deploy a Stack

On the manager node, create a docker-compose.yml (Swarm uses the same format with some additions):

```
version: '3.8'
services:
  api:
    image: 123456789.dkr.ecr.us-east-1.amazonaws.com/docker-app:v1
```

```

ports:
  - "3000:3000"
environment:
  DB_HOST: db
  DB_USER: root
  DB_PASS: MyS3cur3P@ss
  DB_NAME: docker_lab
deploy:
  replicas: 3
  update_config:
    parallelism: 1
    delay: 10s
  restart_policy:
    condition: on-failure

db:
  image: mysql:8
  environment:
    MYSQL_ROOT_PASSWORD: MyS3cur3P@ss
    MYSQL_DATABASE: docker_lab
  volumes:
    - db_data:/var/lib/mysql
  deploy:
    replicas: 1
    placement:
      constraints:
        - node.role == manager

volumes:
  db_data:

```

Note

Before deploying, each node needs access to ECR. Run the ECR login command on ALL 3 nodes, or use IAM instance profiles with ECR pull permissions.

The `deploy.replicas: 3` tells Swarm to run 3 copies of the API across the cluster. Database stays on the manager (placement constraint).

Deploy the stack:

```

docker stack deploy -c docker-compose.yml myapp

# Check services
docker service ls

# Check where containers are running
docker service ps myapp_api

```

7.10 — Test Load Balancing

Swarm has a built-in load balancer called the routing mesh. Hit any node's IP on port 3000:

```
# From your local machine, try all 3 IPs
curl http://<MANAGER-IP>:3000/api/health
curl http://<WORKER-1-IP>:3000/api/health
curl http://<WORKER-2-IP>:3000/api/health
```

All 3 should work, and the container hostname in the response will rotate between the 3 replicas.

7.11 — Scaling & Rolling Updates

```
# Scale up to 5 replicas
docker service scale myapp_api=5

# Watch them come up
docker service ps myapp_api

# Rolling update (after pushing a new image version)
docker service update --image \
  123456789.dkr.ecr.us-east-1.amazonaws.com/docker-app:v2 \
  myapp api
```

Swarm updates containers one at a time (parallelism: 1) with a 10-second delay between each — zero-downtime deployment.

Part 8: Practice — Build Your Own

Now build your own application from scratch, containerize it, and deploy it to your Swarm cluster.

Requirements

- **Choose any language/framework:** Python Flask, Node.js Express, Go, Ruby Sinatra — whatever you want.
- **Your app must have** at least 2 API endpoints + a health check endpoint.
- **Write a Dockerfile** from scratch for your app.
- **Write a docker-compose.yml** with your app + a database (MySQL or PostgreSQL).
- **Push your image to ECR.**
- **Deploy your Swarm cluster** with at least 3 replicas.
- **Demonstrate:** scaling (up and down), rolling update, and load balancing (hit different nodes, get different container hostnames).

Clean Up

```
# Remove the stack
docker stack rm myapp

# Leave the swarm (on workers first, then manager)
docker swarm leave          # on workers
docker swarm leave --force  # on manager

# Terminate your EC2 instances
# Delete the ECR repository
aws ecr delete-repository --repository-name docker-app \
  --force --region us-east-1
```

Warning

Don't forget to terminate your 3 EC2 instances and delete the ECR repository to avoid charges.

Lab Recap

What You Did	Docker Concepts
Pulled and ran Nginx	Images, containers, port mapping
Built a custom image	Dockerfile, layers, build cache
Docker Compose with MySQL	Multi-container apps, networking, volumes, environment variables
Pushed to ECR	Private registries, authentication, tagging
Docker Swarm cluster	Orchestration, services, replicas, routing mesh
Scaling & rolling updates	docker service scale, zero-downtime updates
Built your own app	End-to-end containerization workflow

End of Lab 008 — You can now package, ship, and orchestrate containers. 🐳